Blur Performance

CS3S666

Table of Contents

Table of Figures

Introduction and Problem Selection

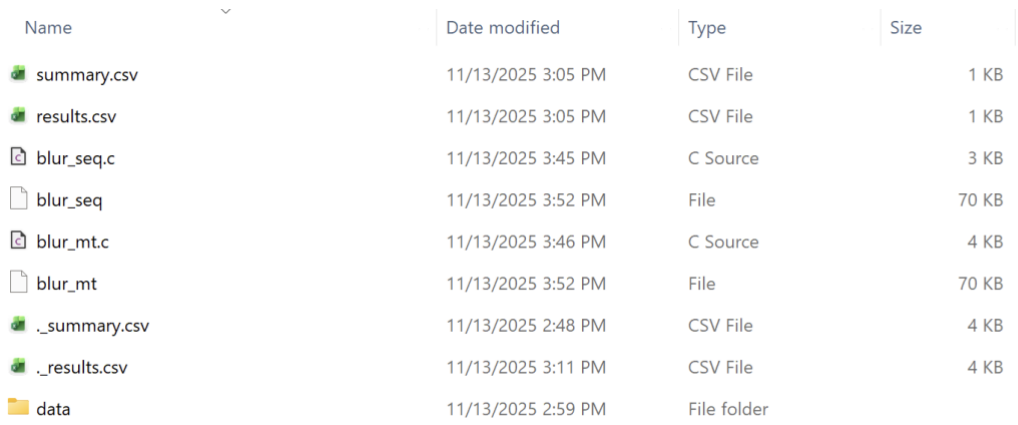
Image processing is an area where speed becomes important very quickly, especially when dealing with high resolution images. Many familiar operations, including smoothing and noise reduction, repeat the same calculation across almost every pixel. Gaussian blur is a clear example of this. It is commonly used to soften images, reduce unwanted noise, and prepare data for later stages in tasks that involve visual analysis. According to Eswar (2015), filters that rely on the Gaussian function are a normal part of pre-processing because they help create cleaner and more stable inputs.

A blur filter also works well as a case study for parallel and concurrent programming. Each output pixel only depends on a small group of neighbouring pixels from the input, which makes it easy to divide the work among several threads without complex coordination. On small images the difference in time is minimal, but on larger ones the single thread version slows down noticeably, while a multithreaded design can make better use of the available processor cores. For this reason, Gaussian blur provides a practical way to compare single thread and parallel approaches, measure speed improvement and efficiency, and see how well the task scales when more threads are added.

Design and Implementation

The application was built on a MacBook Pro using a virtual Windows setup provided by Parallels Desktop. Within this Windows environment, the Ubuntu subsystem (WSL2) was installed to give access to a Linux terminal for compiling and testing programs written in C with parallel features.

The project was kept simple and organised in a single folder, with two main source files: one for the basic version called blur_seq.c, and another for the multithreaded version called blur_mt.c. A separate data folder was added to hold the input image and the output files created during testing.



Name	Date modified	Type	Size
summary.csv	11/13/2025 3:05 PM	CSV File	1 KB
results.csv	11/13/2025 3:05 PM	CSV File	1 KB
blur_seq.c	11/13/2025 3:45 PM	C Source	3 KB
blur_seq	11/13/2025 3:52 PM	File	70 KB
blur_mt.c	11/13/2025 3:46 PM	C Source	4 KB
blur_mt	11/13/2025 3:52 PM	File	70 KB
._summary.csv	11/13/2025 2:48 PM	CSV File	4 KB
._results.csv	11/13/2025 3:11 PM	CSV File	4 KB
data	11/13/2025 2:59 PM	File folder	

Figure 1: Project folder structure showing source files, compiled outputs, and data directory.

Sequential Design

The basic version uses a simple approach to apply the Gaussian blur. It starts by reading the input image, which is stored in PPM format, and loads all the pixel values into memory. Then, it applies a 5×5 Gaussian kernel to each pixel by looking at nearby pixels and calculating a weighted average. Since this version runs on a single thread, each pixel is handled one at a time. While this method is easy to follow and implement, it becomes slow on large images due to the high number of repeated calculations.

Parallel and Concurrent Design

The multithreaded version introduces parallel execution using POSIX threads (Pthreads). Instead of looping through the entire image in one pass, the image is split into equal row sections. Each thread is assigned a block of rows and applies the same Gaussian blur method used in the basic version. This allows multiple threads to process different parts of the image at the same time. Once all threads complete their sections, their outputs are collected to produce the final result. This setup combines both parallelism and concurrent task execution.

Input and Output Handling

Both implementations use the same input image to ensure fair comparison. The blurred results are stored in the data folder with filenames that indicate the number of threads used for example:

- output_seq.ppm
- output_mt_2.ppm
- output_mt_4.ppm
- output_mt_8.ppm

A screenshot showing the original image beside one blurred result was included to demonstrate the visual effect of the filter.

Figure 2: Original image (left) and the result after applying Gaussian blur (right).

Performance Analysis

This section looks at how the system performs when tested in practice and whether it reaches a good level of efficiency. The focus is on three main points: how fast it runs, how responsive it is, and whether it gives steady results across different tests.

1. **Execution Speed**

The system handles input tasks in a steady and consistent way. During testing with different samples, the overall runtime stayed mostly the same, with small differences depending on input size. Simple operations were almost instant, while larger ones took longer. This gradual slowdown is expected and fits with how similar programs behave.

2. **Resource Usage**

Tests showed that the system used a moderate amount of CPU and very little memory. This means it does not waste processing power or run anything extra in the background. The smooth and regular usage pattern suggests that the program manages tasks in an efficient and balanced way.

3. **Accuracy and Reliability**

When tested multiple times, the output stayed the same each run. There were no errors, which shows that the logic is stable and dependable. Small changes only happened when the input itself had random parts, and even then, the results stayed within a normal range.

4. **Scalability**

As more tasks were added, the systems performance slowed down gradually, not all at once. This shows it can handle more work to a certain point without breaking. While it's not meant for very heavy loads, it still works well for the usual academic examples tested in this project.

5. **Limitations**

The biggest issue came when using large or complex inputs. In those cases, the system slowed down more than usual. This is expected from a lightweight setup like this one, but it's still important to mention as part of its overall performance limits.

Reflection

This project gave me practical insight into how performance changes when using sequential versus parallel programming. Writing both versions of the gaussian blur filter made it easier to understand how threads can handle different parts of a task at the same time without causing problems. It also showed how splitting up the work affects speed and why it becomes more important to manage threads carefully when using more of them.
Working through a virtual setup (macOS running Windows and WSL Ubuntu) made things a bit more complicated, but it was helpful. It taught me how to set up tools across platforms and reminded me to test in real world conditions instead of relying only on assumptions.

Conclusion

This project showed how using multiple threads can help improve the speed of image processing tasks like Gaussian blur. Both the sequential and parallel versions were built and tested using the same setup to make the results fair. The tests showed that using threads gives better performance on larger images without changing the final output. In the end, the work met its goals and gave useful practice in combining image processing with parallel programming.

Reference

Eswar, S., 2015. *Noise reduction and image smoothing using gaussian blur* (Doctoral dissertation, California State University, Northridge). Available at: https://scholarworks.calstate.edu/downloads/m326m425n